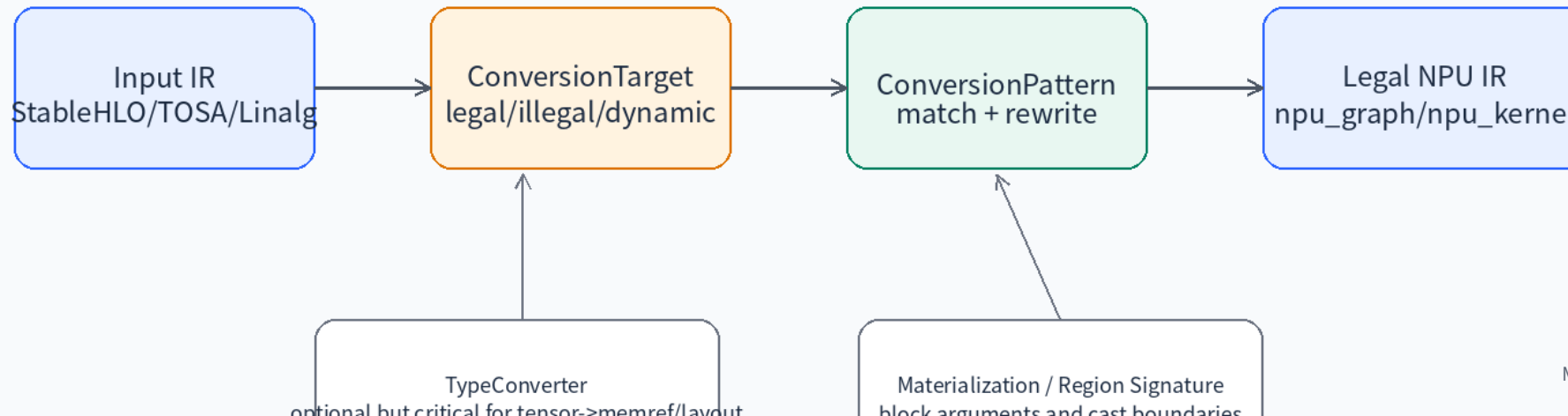


Lecture 11

Dialect Conversion

NPU legal op set 과 unsupported op lowering

Dialect Conversion Flow for NPU Compiler



오늘의 위치

- 09 Pass Infrastructure: pass pipeline 설계
- 10 Rewrite Pattern: local fusion/cleanup
- 11 Dialect Conversion: legal/illegal boundary 정의
- 12 Bufferization: tensor -> memref memory boundary

Learning Objectives

- ConversionTarget 으로 NPU legal op set 을 정의한다
- dynamic legality 로 shape/dtype/layout 제약을 encode 한다
- ConversionPattern 으로 unsupported op 를 낮춘다
- TypeConverter 와 region signature conversion 의 역할을 설명한다
- positive/negative regression test 를 작성한다

왜 Dialect Conversion 인가 ?

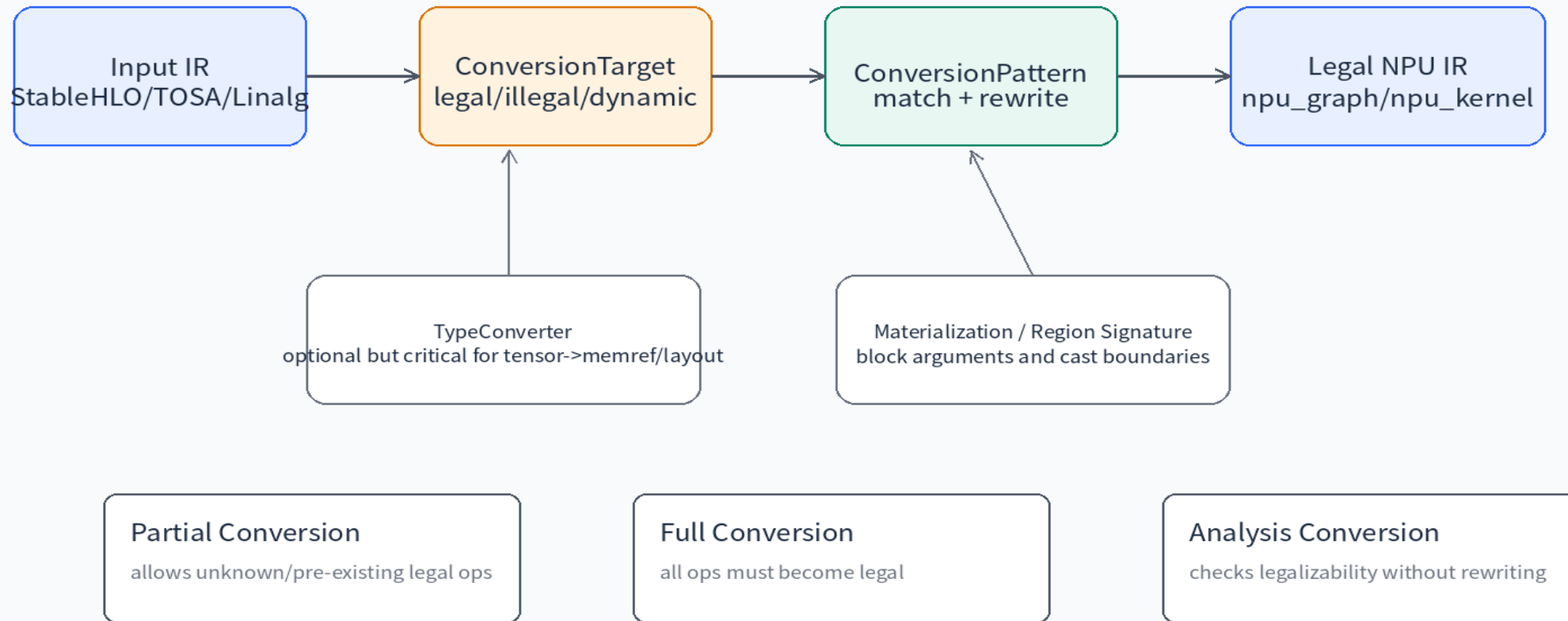
- RewritePattern 은 “바꿀 수 있으면 바꾸기” 에 가깝다
- Dialect Conversion 은 “이 boundary 이후 무엇이 legal 인가 ?” 를 명시한다
- NPU compiler 에서는 target capability contract 가 pass 의 성공 / 실패 기준이 된다

Dialect Conversion 구성 요소

- ConversionTarget: legal/illegal/dynamic legality
- RewritePatternSet: illegal op 를 legal op sequence 로 변환
- TypeConverter: type/block argument/materialization 처리
- applyPartialConversion / applyFullConversion / applyAnalysisConversion

Conversion Flow

Dialect Conversion Flow for NPU Compiler



ConversionTarget = Target Contract

- addLegalDialect: dialect 전체를 legal 로 등록
- addLegalOp: 특정 op 만 legal 로 등록
- addIllegalOp / addIllegalDialect: 반드시 변환되어야 하는 op 지정
- addDynamicallyLegalOp: op property 기반 legality 지정

Static vs Dynamic Legality

- Static legality: op name/dialect 만 보고 판단
- Dynamic legality: shape, dtype, layout, quant attr, target feature 를 검사
- NPU 에서는 대부분 dynamic legality 가 필요하다

Recursive Legality

- 어떤 op 의 region 내부를 “이미 legal 한 island” 로 취급할 수 있다
- 예 : `npu_kernel.launch` 내부 command sequence 를 region 단위로 보호
- 잘못 쓰면 내부 illegal op 가 숨어서 final lowering 을 깨뜨린다

Conversion Mode #1 - Partial

- 명시적으로 illegal 인 op 를 가능한 만큼 legalize
- unknown/pre-existing legal op 는 남을 수 있음
- NPU progressive lowering 에 적합 : graph 일부만 target op 로 변환

Conversion Mode #2 - Full

- 모든 op 가 target 에서 known legal 이어야 성공
- final boundary 에 적합 : runtime/ABI emission 전
- unsupported op 가 남으면 signalPassFailure

Conversion Mode #3 - Analysis

- IR 을 바꾸지 않고 legalizable 여부를 분석
- NPU capability report 와 fallback planning 에 적합
 - “이 모델이 우리 NPU 에서 돌 수 있는가 ?” 를 빠르게 판단

Conversion Modes 비교

Partial vs Full vs Analysis Conversion

Partial

Legalize marked illegal ops
Allow unknown ops to remain
Best for progressive NPU lowering

NPU use:

Full

Everything must be known legal
Fail if any illegal op remains
Best at final boundary before codegen/runtime

NPU use:

Analysis

No mutation
Record what would legalize
Best for capability report / fallback planning

NPU use:

ConversionPattern vs RewritePattern

- ConversionPattern 은 remapped operand adaptor 를 받는다
- TypeConverter 가 operand/result type 을 바꾼 경우 adaptor 를 사용해야 한다
- illegal op 를 legal op 로 바꾸는 것이 목적이다

Adaptor 를 써야 하는 이유

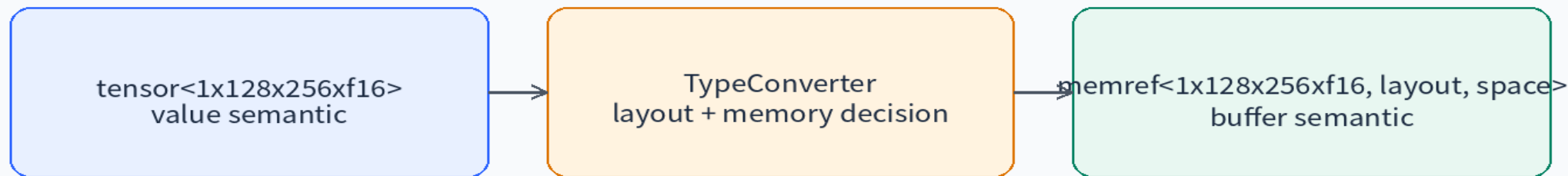
- original operand: old type 일 수 있음
- adaptor operand: conversion framework 가 remap 한 legal type
- NPU layout/type conversion 후 original operand 를 쓰면 verifier failure 가 발생할 수 있다

TypeConverter 핵심

- Type mapping: tensor -> memref/custom descriptor
- Argument conversion: block argument 와 function signature 수정
- Materialization: legal/illegal boundary 에 필요한 cast/bridge op 생성
- NPU 에서는 layout/memory-space/ABI contract 와 연결된다

Type Conversion 예시

Type Conversion and Region Signature Conversion



Before conversion:

```
^bb0(%arg0: tensor<...>, %arg1: tensor<...>):
```

After region signature conversion:

```
^bb0(%arg0: memref<..., #npu.packed, 1>, %arg1: memref<..., #npu.packed, 1>):
```

NPU rule: type conversion is not just syntax. It records layout, memory space, alignment, and ABI contract.

NPU Legal Op Set 은 이름 목록이 아니다

- op name + shape policy + dtype policy + layout policy + target feature
- latency/energy/SRAM fit 조건도 compiler decision 에 반영
- legal op set 은 compiler 와 hardware microarchitecture 의 contract 이다

Legal Op Set Matrix

NPU Legal Op Set Audit Matrix

Op class	Shape policy	DType policy	Layout policy	HW mapping	Decision
matmul	static M/N/K $K \% 32 == 0$	i8/u8/f16 acc i32/f32	A/B packed C row-major	MMA array	Legal or dynamic legal
conv2d	static kernel stride/dilation bounded	i8/f16	NHWC/HWCF or packed	im2col/direct	Legal with constraints
elementwise	broadcast limited	i8/f16/f32	same layout	SIMD/ALU	Legal
reduction	axis static rank $\leq$ policy	acc wider	contiguous axis preferred	tree reduce	Legalize/decompose
gather/scatter	indices/static? coalescing?	i8/f16/f32	layout sensitive	DMA/indirect	Maybe fallback
exp/tanh/erf	approx needed	f16/i8 LUT	vector-friendly	LUT/poly	Approx or reject

MatMul Dynamic Legality 예시

- M/N/K static
- K multiple matches MMA tile
- A/B layout is packed or packable
- accumulator type is i32/f32
- epilogue kind is supported: none/bias/relu/gelu_lut

Conv2D Dynamic Legality 예시

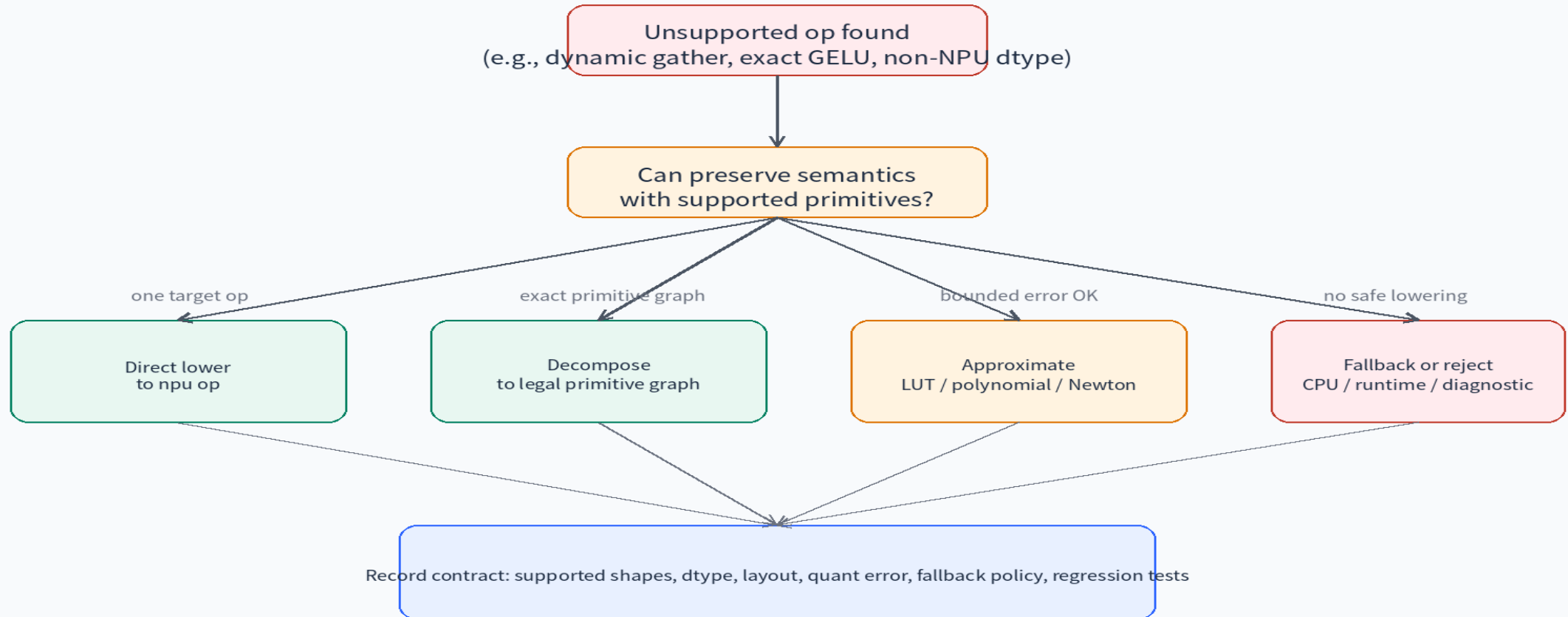
- NHWC/HWCF or compiler-supported packed layout
- kernel/stride/dilation bounds satisfied
- group/depthwise support checked separately
- SRAM footprint fits tile policy

Reduction Dynamic Legality 예시

- axis must be static
- rank and reduction length within hardware policy
- accumulator precision defined
- reduction axis should be contiguous or cheaply transposable

Unsupported Op Decision Tree

Unsupported Operation Lowering Decision Tree



Case Study A - GELU

- Exact erf/tanh may be unsupported on edge NPU
- Lowering options: tanh polynomial, LUT, piecewise linear, CPU fallback
- Compiler must record accuracy budget and calibration domain

Case Study B - LayerNorm rsqrt

- rsqrt may be approximated by LUT + Newton iteration
- epsilon placement affects numerical compatibility
- fusion with preceding/following ops can reduce DRAM traffic

Case Study C - Gather/Scatter

- Irregular memory access may break DMA coalescing
- If indices are static: precompute or transform layout
- If dynamic and uncoalesced: fallback may be better than NPU execution

Pass Skeleton 구조

- Define ConversionTarget
- Mark legal/illegal/dynamic legal ops
- Populate conversion patterns
- Apply partial/full conversion
- signalPassFailure on illegal leftovers

C++ Pseudocode

- `ConversionTarget target(ctx);`
- `target.addLegalDialect<npu_graph::NPUGraphDialect>();`
- `target.addDynamicallyLegalOp<npu_graph::MatMulOp>(predicate);`
- `patterns.add<LowerUnsupportedGelu, LowerLayerNorm>(typeConverter, ctx);`
- `applyPartialConversion(module, target, patterns);`

Negative Diagnostics

- Unsupported dtype: emit clear message
- Unsupported dynamic shape: tell which dimension is dynamic
- Unsupported layout: suggest pack/transpose pass
- Unsupported accuracy: require fallback or calibration flag

FileCheck Strategy

- CHECK-NOT: unsupported op should disappear
- CHECK: target npu op appears
- CHECK-SAME: important attrs preserved
- negative test: -verify-diagnostics for unsupported case

Integration in NPU Pipeline

- After canonicalize/decompose/fusion
- Before bufferization when tensor-level semantics are still available
- May run multiple times: graph legalization, kernel legalization, runtime legalization

Common Mistakes

- Using RewritePattern where conversion legality is needed
- Declaring entire dialect legal too early
- Forgetting dynamic legality predicates
- Using original operands instead of adaptor operands
- No negative tests for unsupported model cases

Exercise A

- Fill NPU legal op set table for matmul/conv/reduce/gather/gelu
- Mark each as legal, dynamic legal, decompose, approximate, or fallback

Exercise B

- Write ConversionTarget skeleton for npu_graph legalization
- Add at least two dynamic legality predicates
- Define unsupported op diagnostics

Exercise C

- Lower MatMul + Bias + unsupported GELU to `npu_graph.matmul_epilogue` with `gelu_lut`
- Write before/after MLIR and FileCheck expectations

Takeaways

- Dialect Conversion turns target capability into compiler-enforced legality
- NPU legal op set must include shape/dtype/layout/quant/memory policy
- Unsupported op handling is a product decision: exact, approximate, fallback, or reject
- Good diagnostics and negative tests are mandatory for production NPU compilers

References

- MLIR Dialect Conversion
- MLIR Toy Tutorial Ch.5/Ch.6
- MLIR Language Reference
- MLIR Pattern Rewriter